UNIT-5

EXCEPTION HANDLING AND MULTITHREADED PROGRAMMING

5.1 TYPES OF ERROR

1.Compile-time Errors:

- Compile-time errors, also known as compilation errors or syntax errors, occur during the compilation phase of the program.
- These errors are caused by violations of the Java syntax rules or incorrect usage of language constructs.
- Examples of compile-time errors include missing semicolons, undefined variables, incompatible types, or using a reserved keyword as an identifier.
- When a compile-time error occurs, the compiler generates an error message, and the program cannot be compiled until the errors are fixed.

2.Runtime Errors:

- Runtime errors, also called exceptions, occur during the execution of the program.
- These errors are caused by exceptional conditions or unexpected situations that arise while the program is running.
- Common examples of runtime errors include division by zero (**ArithmeticException**), accessing an array element with an invalid index (**ArrayIndexOutOfBoundsException**), or attempting to open a non-existent file (**FileNotFoundException**).
- When a runtime error occurs, an exception object is thrown, which can be caught and handled using exception handling mechanisms like try-catch blocks. If unhandled, the program terminates abruptly.

3.Logical Errors:

- Logical errors, also known as semantic errors or bugs, occur when the program's logic or algorithm is incorrect.
- These errors do not cause the program to crash or generate error messages but result in undesired or incorrect behavior.
- Logical errors can be challenging to identify as they do not produce any compile-time or runtime errors. The program runs successfully, but the output or behavior is not as expected.
- Detecting and fixing logical errors typically involves careful code review, debugging, and testing**.**

It is important to note that compile-time errors are caught by the compiler during the compilation process, runtime errors occur during program execution and can be caught and handled using exception handling, while logical errors require careful analysis and debugging to identify and fix.

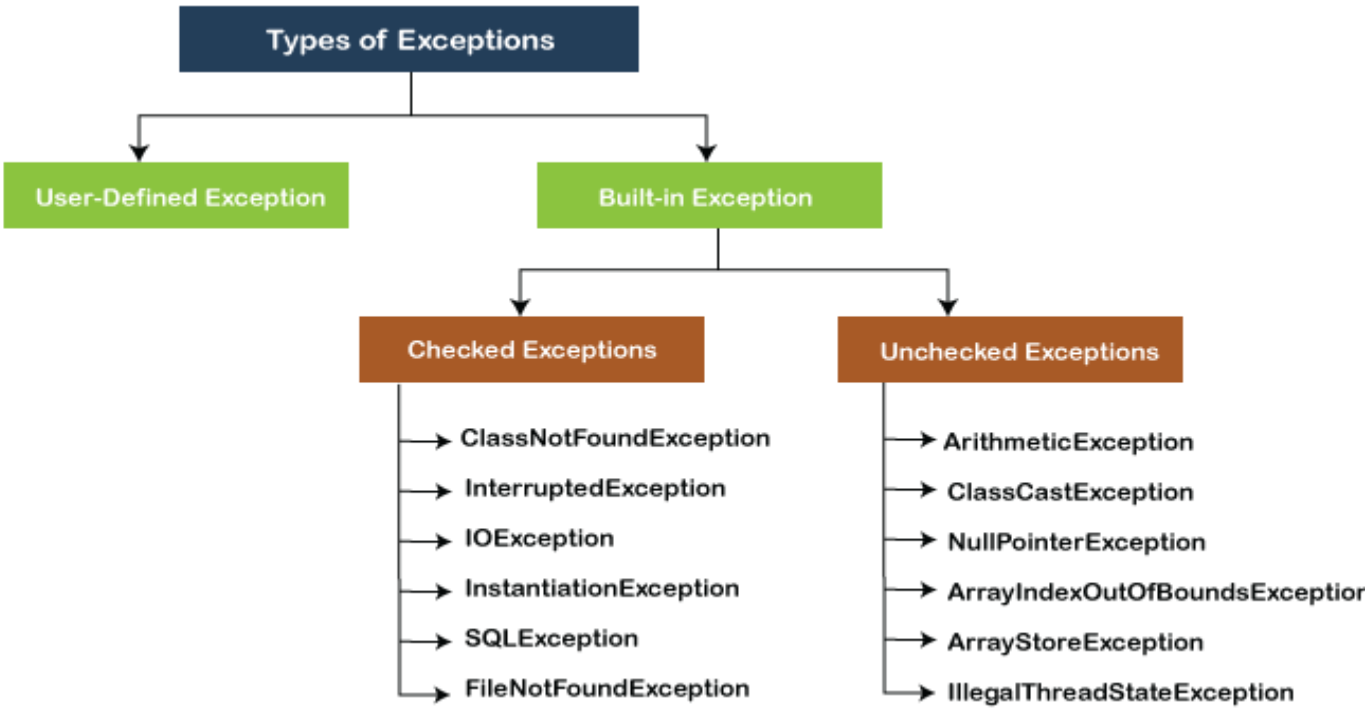
5.2 BASIC CONCEPTS OF EXCEPTION HANDLING

Exception handling in Java is a mechanism that allows you to handle and recover from exceptional situations or errors that occur during the execution of a program. It helps in making the program more robust by providing a structured approach to deal with unexpected or exceptional scenarios.

Exception

- An exception is an event that occurs during the execution of a program and disrupts the normal flow of the program.
- Exceptions represent various types of errors, such as runtime errors, input/output errors, arithmetic errors, and others.
- In Java, exceptions are represented by classes that are derived from the Throwable class or one of its subclasses.

Types of Exception:



1. Built-in Exception

Exceptions that are already available in **Java libraries** are referred to as **built-in exception.**

Checked Exception:

Checked exceptions are called **compile-time** exceptions because these exceptions are checked at compile-time by the compiler.

EXAMPLE:

```
import java.io.*;
class CheckedExceptionExample
{
```

```
   public static void main(String args[])
{
      FileInputStream file_data = null;
      file_data = new FileInputStream("C:/Users/ajeet/OneDrive/Desktop/Hello.txt");
      int m;
      while(( m = file_data.read() ) != -1)
         {
         System.out.print((char)m);
      }
      file_data.close();
   }
}
```

Unchecked Exception:

The **unchecked** exceptions are just opposite to the **checked** exceptions. The compiler will not check these exceptions at compile time. In simple words, if a program throws an unchecked exception, and even if we didn't handle or declare it, the program would not give a compilation error. Usually, it occurs when the user provides bad data during the interaction with the program.

EXAMPLE:

```
        class UncheckedExceptionExample1
        {
  public static void main(String args[])
  {
   int postive = 35;
   int zero = 0;
   int result = positive/zero;
   //Give Unchecked Exception here.
System.out.println(result);
  }
}
```

2. User-defined Exception

we can write our own exception class by extends the Exception class. We can throw our own exception on a particular condition using the throw keyword. For creating a user-defined exception, we should have basic knowledge of **the try-catch** block and **throw** keyword.

EXAMPLE:

```
import java.util.*;
class UserDefinedException
{
   public static void main(String args[])
{
      try{
         throw new NewException(5);
      }
      catch(NewException ex)
{
         System.out.println(ex);
      }
   }
}
class NewException extends Exception
{
   int x;
   NewException(int y)
{
      x=y;
   }
   public String toString()
{
      return ("Exception value =  "+x) ;
   }
}
```

5.3 TRY AND CATCH BLOCK

The try-catch block is used to handle exceptions and provide a mechanism to gracefully handle exceptional situations during the execution of a program. The try-catch block consists of a try block and one or more catch blocks.

```
try {
  // Code that may throw an exception
} catch (ExceptionType1 ex1) {
 // Exception handling code for ExceptionType1
} catch (ExceptionType2 ex2) {
// Exception handling code for ExceptionType2
} catch (Exception ex) {
  // Generic exception handling code for any other exception
}
```

If an exception occurs within the try block, the control immediately transfers to the corresponding catch block based on the type of exception.

Each catch block specifies the type of exception it can handle. If the caught exception matches the type specified in a catch block, that catch block is executed.

If no catch block is found that matches the type of the thrown exception, the exception propagates to the next level of the method call stack or terminates the program if it reaches the main method without being caught.

EXAMPLE:

```
public class Main
{
 public static void main(String[ ] args)
{
 try {
    int[] myNumbers = {1, 2, 3};
   System.out.println(myNumbers[10]);
  } catch (Exception e) {
  System.out.println("Something went wrong.");
    }
}
}
```

5.3.1 Multiple Catch Block

EXAMPLE:

```
public class MultipleCatchBlock1
{

  public static void main(String[] args)
   {

    try
    {
      int a[]=new int[5];
      a[5]=30/0;
     }
     catch(ArithmeticException e)
       {
        System.out.println("Arithmetic Exception occurs");
       }
     catch(ArrayIndexOutOfBoundsException e)
       {
        System.out.println("ArrayIndexOutOfBounds Exception occurs");
       }
     catch(Exception e)
       {
        System.out.println("Parent Exception occurs");
       }
     System.out.println("rest of the code");
  }
}
```

5.4 THROW AND THROWS

throw and **throws** are two keywords used in exception handling to deal with exceptions and specify exception propagation.

1.throw Keyword:

- The throw keyword is used to explicitly throw an exception from within a method or a block of code.
- It is followed by an instance of an exception class or a subclass of Throwable.
- When a throw statement is encountered, the normal flow of the program is disrupted, and the specified exception is thrown.
- It allows you to generate and throw exceptions explicitly to handle exceptional scenarios.

EXAMPLE:

```
public class TestThrow1
{
//function to check if person is eligible to vote or not
 public static void validate(int age)
{
   if(age<18)
{
   //throw Arithmetic exception if not eligible to vote
  throw new ArithmeticException("Person is not eligible to vote");
  }
  else {
   System.out.println("Person is eligible to vote!!");
  }
   }
 //main method
   public static void main(String args[])
{
    //calling the function
      validate(13);
     System.out.println("rest of the code...");
  }
}
```

2.throws Keyword:

- The throws keyword is used in a method declaration to indicate that the method may throw one or more types of exceptions.
- It specifies that the method is not handling the exception itself but is instead passing the responsibility to the calling method or the JVM.
- Multiple exceptions can be declared using a comma-separated list.

EXAMPLE:

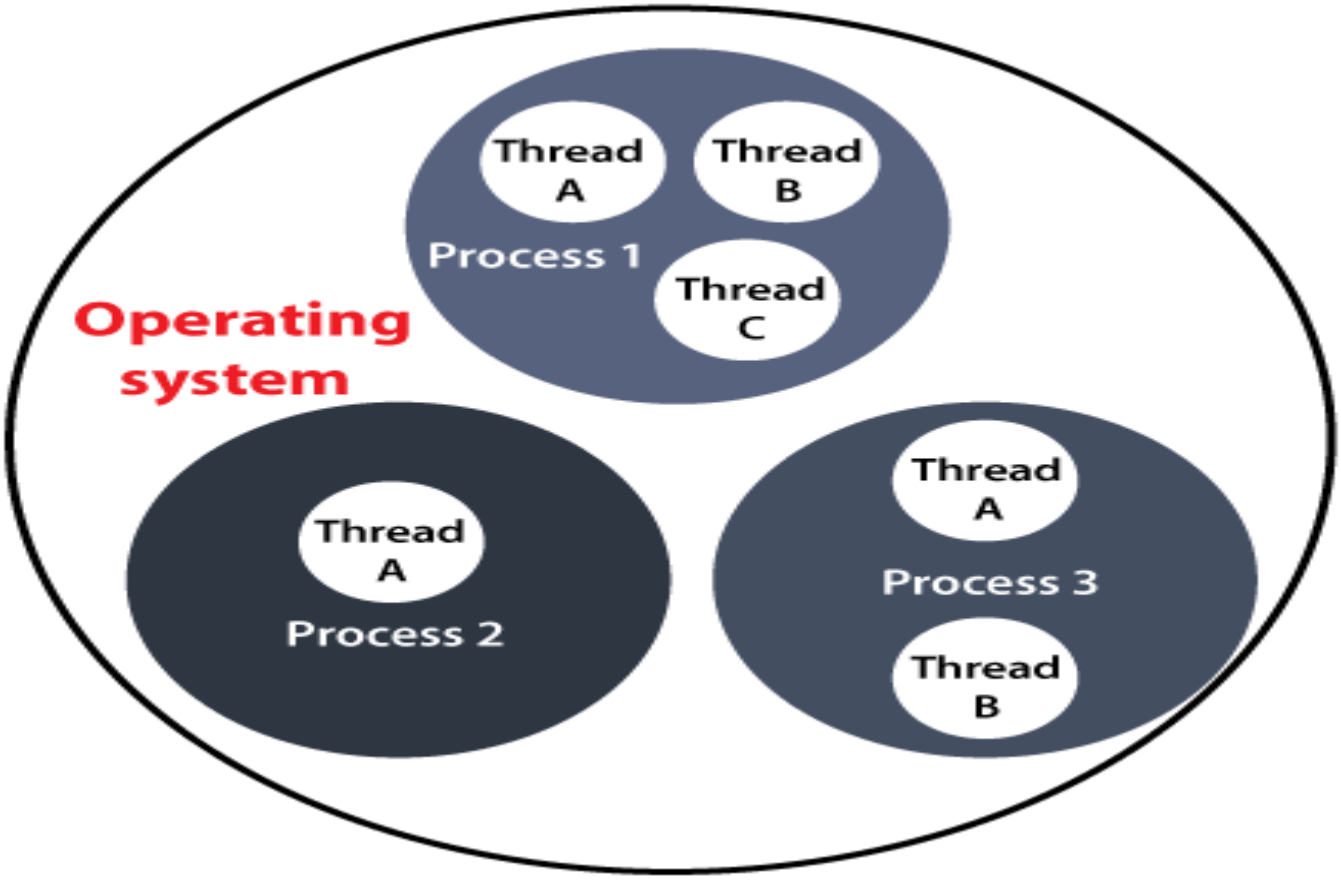
```
import java.io.*;
class M{
void method()throws IOException
{
  throw new IOException("device error");
 }
}
public class Testthrows2
{
   public static void main(String args[])
{
   try{
  M m=new M();
   m.method();
   }
catch(Exception e){System.out.println("exception handled");
}

    System.out.println("normal flow...");
  }
}
```

It's important to note that throw is used to raise an exception explicitly within a method, whereas **throws** is used to indicate that a method may potentially **throw** an exception, allowing the caller to handle it or propagate it further.

5.5 INTRODUCTION OF THREAD

A Thread is a very light-weighted process, or we can say the smallest part of the process that allows a program to operate more efficiently by running multiple tasks simultaneously.

In order to perform complicated tasks in the background, we used the Thread concept in Java. All the tasks are executed without affecting the main program. In a program or process, all the threads have their own separate path for execution, so each thread of a process is independent.



5.6 IMPLEMENTATION OF THREAD

In Java, there are two primary ways to implement threads: by extending the **Thread** class or by implementing the **Runnable** interface.

1.Extending the Thread class:

In this approach, you create a new class that extends the **Thread** class and override its **run**() method, which represents the code that will be executed in the new thread.

You can then create an instance of your custom thread class and start the thread by calling the **start**() method.

EXAMPLE:

```
class MyThread extends Thread {
   @Override
 public void run() {
```

```
 // Code to be executed in the new thread
  System.out.println("Thread is running");
 }
}

public class Main {
  public static void main(String[] args) {
    MyThread thread = new MyThread();
thread.start(); // Starts the execution of the new thread
   }
}
```

2.Implementing the Runnable interface:

In this approach, you create a class that implements the **Runnable** interface and implement the **run**() method defined by the interface.

You then create an instance of your custom class and pass it to a **Thread** object's constructor.

EXAMPLE:

```
class MyRunnable implements Runnable {
   @Override
   public void run() {
    // Code to be executed in the new thread
  System.out.println("Thread is running");
 }
}

public class Main {
  public static void main(String[] args) {
   MyRunnable runnable = new MyRunnable();
   Thread thread = new Thread(runnable);
thread.start(); // Starts the execution of the new thread
  } }
```

Both approaches create a new thread of execution, and the run() method is called when the thread starts. You can perform any desired operations within the run() method. The start() method initiates the execution of the thread, which will run concurrently with the main thread.

5.7 THREAD LIFE CYCLE AND METHOD

The life cycle of a thread in Java refers to its various states and transitions that occur during its execution. A thread goes through several stages from its creation to termination.

The Java thread life cycle consists of the following states:

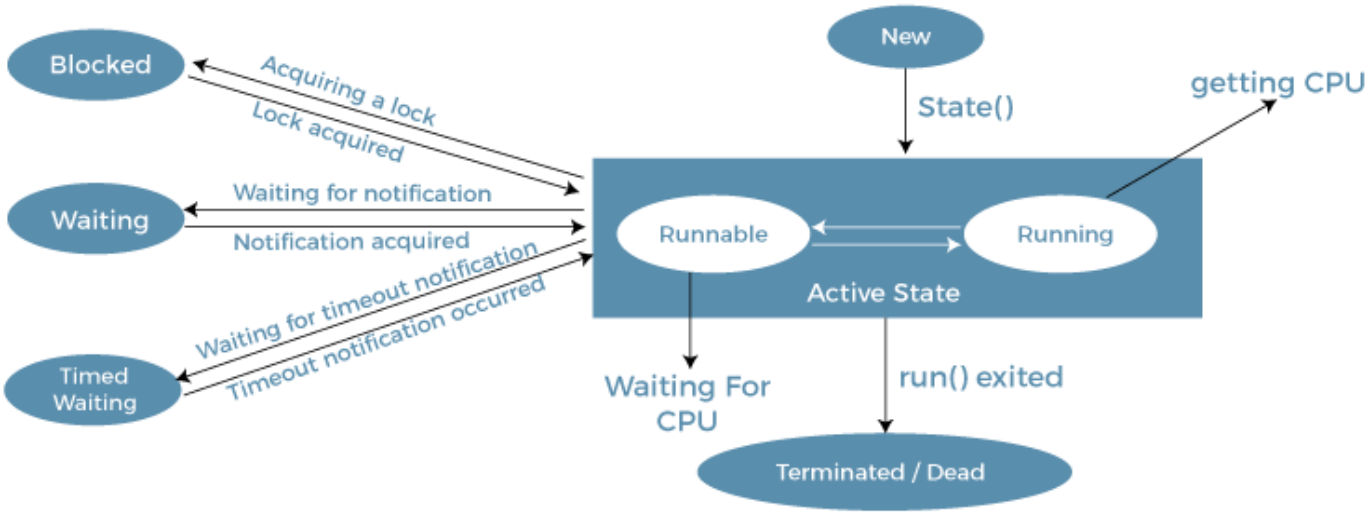
New

Active

Blocked / Waiting

Timed Waiting

Terminated



Life Cycle of a Thread

1.New:

The thread is in the new state when it has been created but has not yet started.

In this state, the thread has been instantiated but has not yet invoked the **start**() method.

2.Runnable:

When the **start**() method is called, the thread enters the runnable state.

In this state, the thread is eligible for execution, but it may or may not be currently executing.

The thread scheduler selects a thread from the runnable pool and executes it.

3.Running:

A thread enters the running state when it starts executing its **run**() method.

In this state, the actual execution of the thread's code is taking place.

The thread remains in this state until it is paused, interrupted, or its execution completes.

4.Blocked/Waiting:

A thread can transition to the blocked or waiting state for various reasons:

Blocked: If the thread is waiting for a monitor lock to enter a synchronized block or method and another thread holds the lock.

Waiting: If the thread is waiting indefinitely for another thread to perform a particular action, such as waiting for a condition to be satisfied or waiting for a **notify**/**notifyAll** signal.

5.Timed Waiting:

Similar to the blocked/waiting state, a thread can also enter a timed waiting state, where it waits for a specific period of time.

This state is reached when the thread calls methods such as **Thread.sleep(), Object.wait()**, or **Thread.join()** with a specified timeout.

6.Terminated:

A thread enters the terminated state when its **run**() method completes execution or when it is terminated prematurely.

5.8 MULTITHREADING

Multithreading in Java refers to the concurrent execution of multiple threads within a single program. It allows different threads to run independently, performing tasks simultaneously and improving overall application performance and responsiveness. Java provides built-in features and APIs to support multithreading, making it easier to develop concurrent applications.

EXAMPLE:

```
        class MultithreadingDemo extends Thread
        {
  public void run()
  {
     try {
        // Displaying the thread that is running
        System.out.println(
           "Thread " + Thread.currentThread().getId()
           + " is running");
     }
     catch (Exception e) {
        // Throwing an exception
        System.out.println("Exception is caught");
     }
```

```
    }
}

// Main Class
public class Multithread {
    public static void main(String[] args)
    {
        int n = 8; // Number of threads
        for (int i = 0; i < n; i++) {
            MultithreadingDemo object
                = new MultithreadingDemo();
            object.start();
        }
    }
}
```